	AIR UNIVERSITY
	DEPARTMENT OF ELECTRICAL AND COMPUTER ENGINEERING
	EXPERIMENT NO 14

Lab Title: **AVL TREE IN DSA**

Student Name: **Muammad Waqas** **Reg. No:** **240756**

Objective:Inertion and deletion of nodes AVL tree and then also balance the balance factor(0,1-1) . . . .

.

LAB ASSESSMENT:

Attributes	Excellent (5)	Good (4)	Average (3)	Satisfactory (2)	Unsatisfactory (1)
Ability to Conduct Experiment					
Ability to assimilate the results					
Effective use of lab equipment and follows the lab safety rules					

Total Marks: _____ Obtained Marks: _____

LAB REPORT ASSESSMENT:

Attributes	Excellent (5)	Good (4)	Average (3)	Satisfactory (2)	Unsatisfactory (1)
Data presentation					
Experimental results					
Conclusion					

Total Marks: _____ Obtained Marks: _____

Date: _____ Signature: _____

Department of Computer Engineering

Data Structures And Algorithms

LAB REPORT : 14

Tittle : AVL TREE

Name : Muammad waqas

Roll No: 240756

Submitted To : CE . Engr Maryam

Date : 31 December 24

Objective:

=======(PROBLEM STATEMENT)=====

1. Implement an AVL tree by inserting nodes. Tree should be up to 8 levels. The depth of right tree should be more than left tree.
2. Search the lowest element in the AVL search tree.
3. Delete the 4th level node in AVL tree and link the rest of the tree

CODE

```
#include <iostream>
using namespace std;
// AVL Tree Node
class Node {
public:
int data;
Node* left;
Node* right;
int height;
Node(int val) {
data = val;
left = right = NULL;
height = 1;
}
};
class AVLTree {
private:
// Get height of node
int getHeight(Node* node) {
```

CODE

```
20  if (node == NULL)
21  return 0;
22  return node->height;
23  }
24  // Get balance factor
25  int getBalance(Node* node) {
26  if (node == NULL)
27  return 0;
28  return getHeight(node->left) - getHeight(node->right);
29  }
30  // Right Rotation
31  Node* rightRotate(Node* y) {
32  Node* x = y->left;
33  Node* T2 = x->right;
34  x->right = y;
35  y->left = T2;
36  y->height = max(getHeight(y->left), getHeight(y->right)) + 1
37  x->height = max(getHeight(x->left), getHeight(x->right)) + 1
38  return x;
```

CODE

```
40  // Left Rotation
41  Node* leftRotate(Node* x) {
42  Node* y = x->right;
43  Node* T2 = y->left;
44  y->left = x;
45  x->right = T2;
46  x->height = max(getHeight(x->left), getHeight(x->right)) + 1;
47  y->height = max(getHeight(y->left), getHeight(y->right)) + 1;
48  return y;
49  }
50  // Insert node
51  Node* insert(Node* node, int key) {
52  if (node == NULL)
53  return new Node(key);
54  if (key < node->data)
55  node->left = insert(node->left, key);
56  else
57  node->right = insert(node->right, key);
58  node->height = 1 + max(getHeight(node->left),
```

CODE

```
59  getHeight(node->right));
60  int balance = getBalance(node);
61  // LL Case
62  if (balance > 1 && key < node->left->data)
63  return rightRotate(node);
64  // RR Case
65  if (balance < -1 && key > node->right->data)
66  return leftRotate(node);
67  // LR Case
68  if (balance > 1 && key > node->left->data) {
69  node->left = leftRotate(node->left);
70  return rightRotate(node);
71  }
72  // RL Case
73  if (balance < -1 && key < node->right->data) {
74  node->right = rightRotate(node->right);
75  return leftRotate(node);
76  }
77  return node;
```

CODE

```
79  // Find minimum value node
80  Node* minValueNode(Node* node) {
81  Node* current = node;
82  while (current->left != NULL)
83  current = current->left;
84  return current;
85  }
86  // Delete node
87  Node* deleteNode(Node* root, int key) {
88  if (root == NULL)
89  return root;
90  if (key < root->data)
91  root->left = deleteNode(root->left, key);
92  else if (key > root->data)
93  root->right = deleteNode(root->right, key);
94  else {
95  if ((root->left == NULL) || (root->right == NULL)) {
96  Node* temp = root->left ? root->left : root->right;
97  if (temp == NULL) {
```

CODE

```
 98  temp = root;
 99  root = NULL;
100  } else
101  *root = *temp;
102  delete temp;
103  } else {
104  Node* temp = minValueNode(root->right);
105  root->data = temp->data;
106  root->right = deleteNode(root->right, temp->data);}}
107  if (root == NULL)
108  return root;
109  root->height = 1 + max(getHeight(root->left),
110  getHeight(root->right));
111  int balance = getBalance(root);
112  // Balancing after deletion
113  if (balance > 1 && getBalance(root->left) >= 0)
114  return rightRotate(root);
115  if (balance > 1 && getBalance(root->left) < 0) {
116  root->left = leftRotate(root->left);
```

CODE

```
117 return rightRotate(root);}
118 if (balance < -1 && getBalance(root->right) <= 0)
119 return leftRotate(root);
120 if (balance < -1 && getBalance(root->right) > 0) {
121 root->right = rightRotate(root->right);
122 return leftRotate(root);}
123 return root;}
124 // Inorder Traversal
125 void inorder(Node* root) {
126 if (root != NULL) {
127 inorder(root->left);
128 cout << root->data << " ";
129 inorder(root->right);}}
130 // Delete 4th level node
131 Node* deleteLevel(Node* root, int level) {
132 if (root == NULL)
133 return NULL;
134 if (level == 4) {
135 return deleteNode(root, root->data);
```

CODE

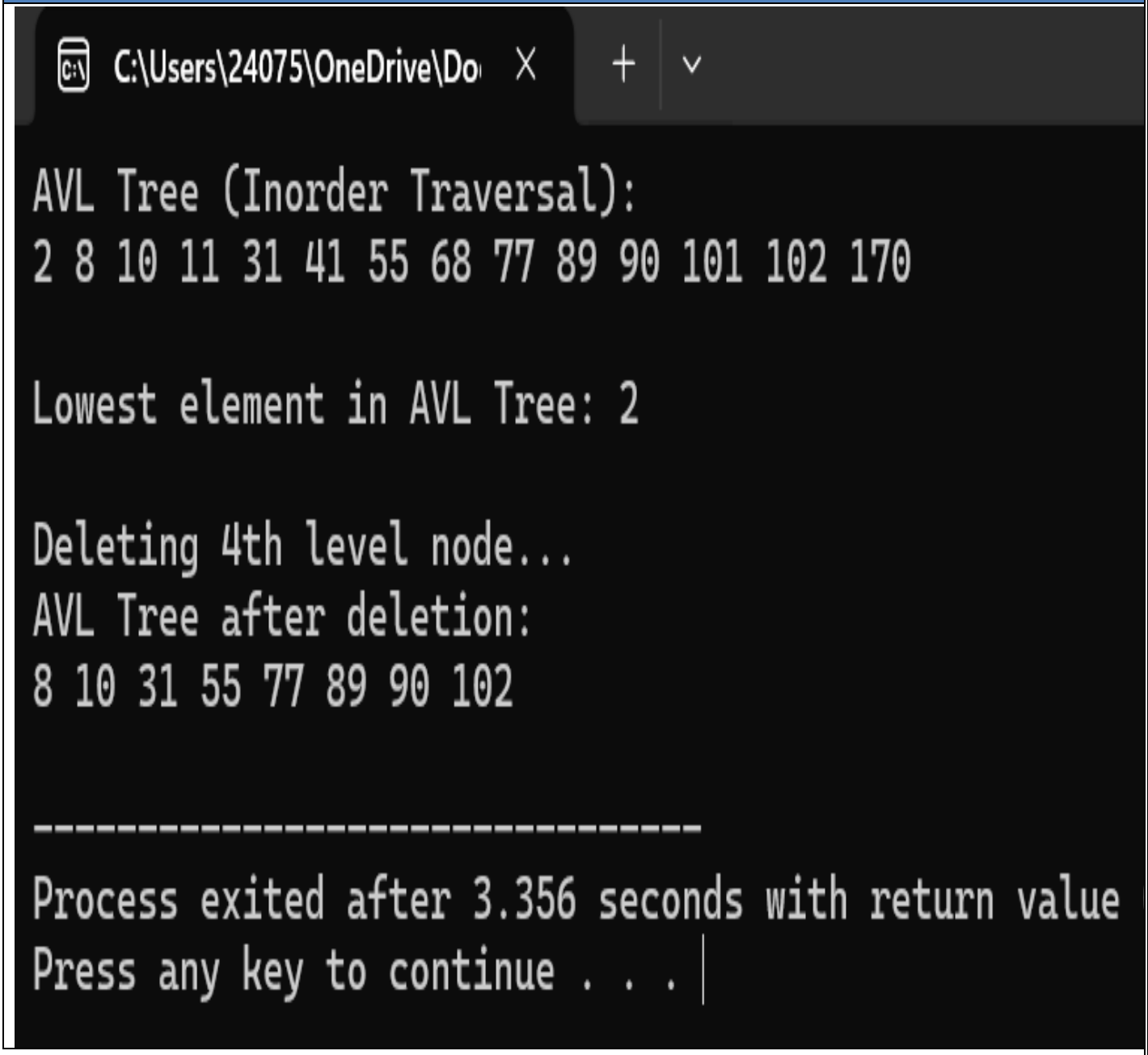
```
137     root->left = deleteLevel(root->left, level + 1);
138     root->right = deleteLevel(root->right, level + 1);
139     return root;
140     }
141     public:
142     Node* root;
143     AVLTree() {
144     root = NULL;
145     }
146     void insertValue(int val) {
147     root = insert(root, val);
148     }
149     void display() {
150     inorder(root);
151     cout << endl;
152     }
153     int findLowest() {
154     Node* temp = minValueNode(root);
```

CODE

```
154 Node* temp = minValueNode(root);
155 return temp->data;
156 }
157 void deleteFourthLevel() {
158 root = deleteLevel(root, 1);
159 }};
160 int main() {
161 AVLTree tree;
162 // Insertion order ensures right subtree deeper than left
163 int values[] = {8,2,11,31,41,55,68,77,89,90,101,102,170,10};
164 for (int i = 0; i < 14; i++)
165 tree.insertValue(values[i]);
166 cout << "AVL Tree (Inorder Traversal):\n";
167 tree.display();
168 cout << "\nLowest element in AVL Tree: ";
169 cout << tree.findLowest() << endl;
170 cout << "\nDeleting 4th level node...\n";
171 tree.deleteFourthLevel();
172 cout << "AVL Tree after deletion:\n";
```

```
173     tree.display();
174     return 0;}
175
```

OUTPUT



❖ Conclusion:

- ❖ The AVL tree maintains balance automatically after every insertion and deletion to ensure efficient searching.
- ❖ Self-balancing is achieved using rotations, which prevent the tree from becoming skewed.
- ❖ The constructed tree reaches up to eight levels while keeping the right subtree deeper than the left subtree as required.
- ❖ Inorder traversal confirms that the AVL tree preserves the binary search tree property.
- ❖ The lowest element in the tree is efficiently found by traversing the leftmost path.
- ❖ Deleting a node at the fourth level correctly reconnects the remaining nodes without breaking the tree structure.
- ❖ After deletion, the tree is rebalanced to maintain logarithmic height and performance.
- ❖ Overall, the implementation demonstrates correct insertion, searching, deletion, and balancing operations in an AVL tree.